
Курс: Рекомендательные системы

Занятие 8:

Генеративные подходы

Лашинин Олег
04.03.26

План занятия



- Генеративные модели
- Авторегрессионный трансформер
- Проблемы dot-product
- Подход к генеративным Next-k генерациям
- RL optimization
- Концепция Semantic-IDS

Почему сегодня эта тема?



Про эту тему больше всего интереса в RecSys R&D
прямо сейчас

Генеративные модели

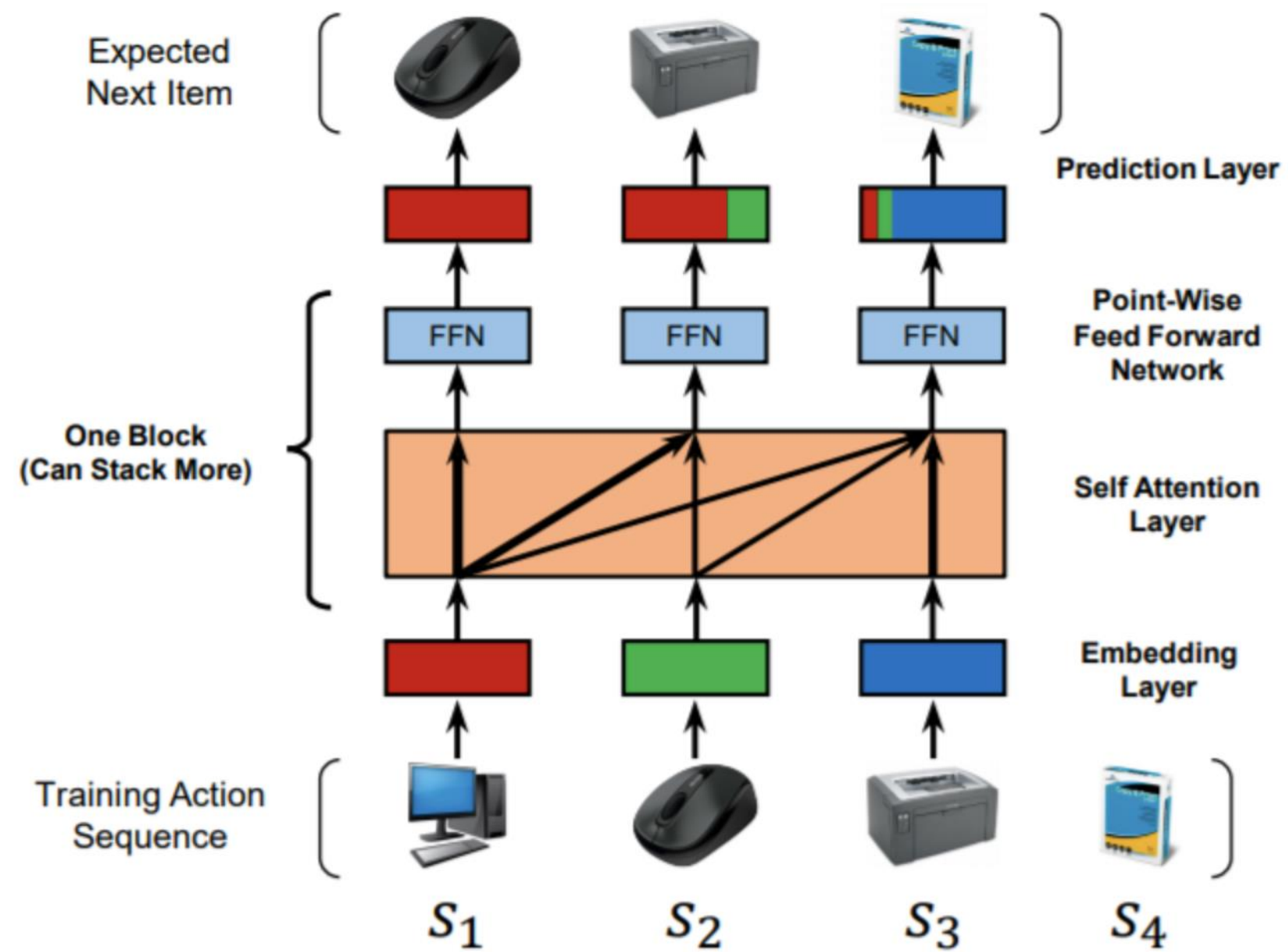


Трансформеры

Вариационные
автоэнкодеры

Диффузионные
модели

Генеративные модели



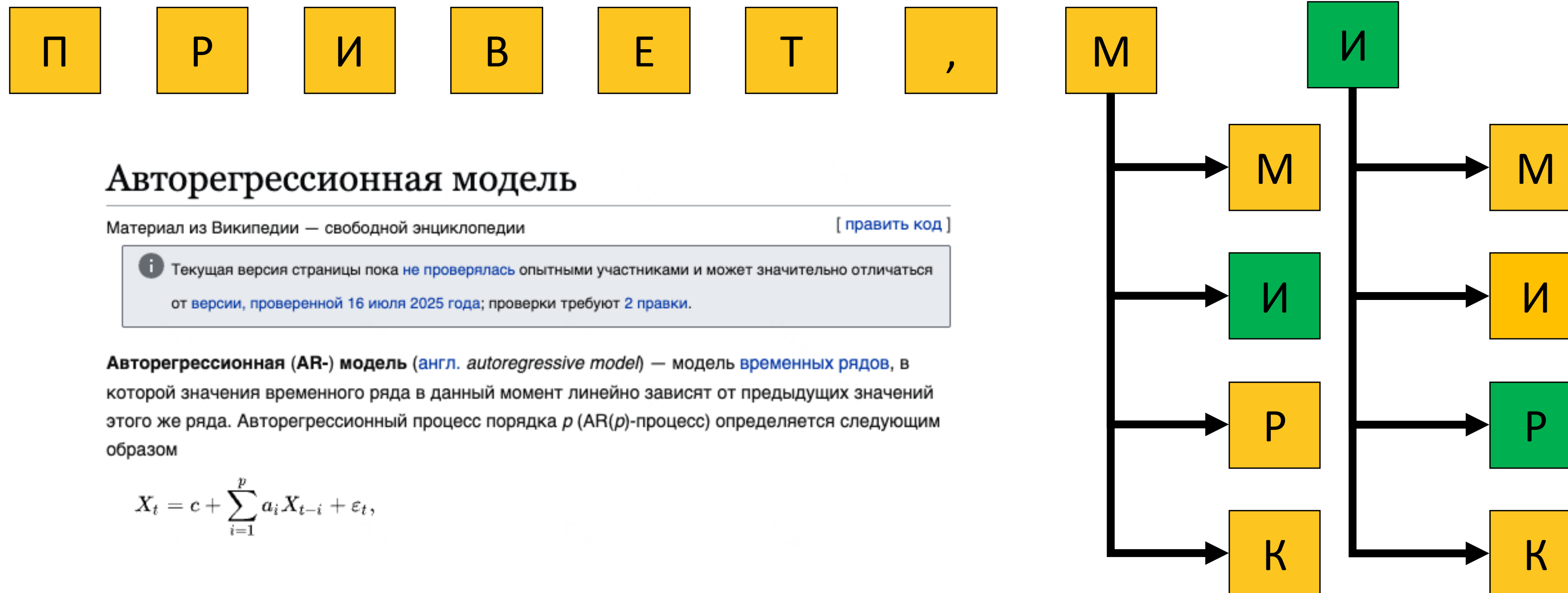
Вспомним SASRec

Figure 1: A simplified diagram showing the training process of SASRec. At each time step, the model considers all previous items, and uses attention to ‘focus on’ items relevant to the next action.

Генеративные модели

П Р И В Е Т , М И Р

Transformer



Авторегрессионная модель

Материал из Википедии — свободной энциклопедии

[\[править код \]](#)

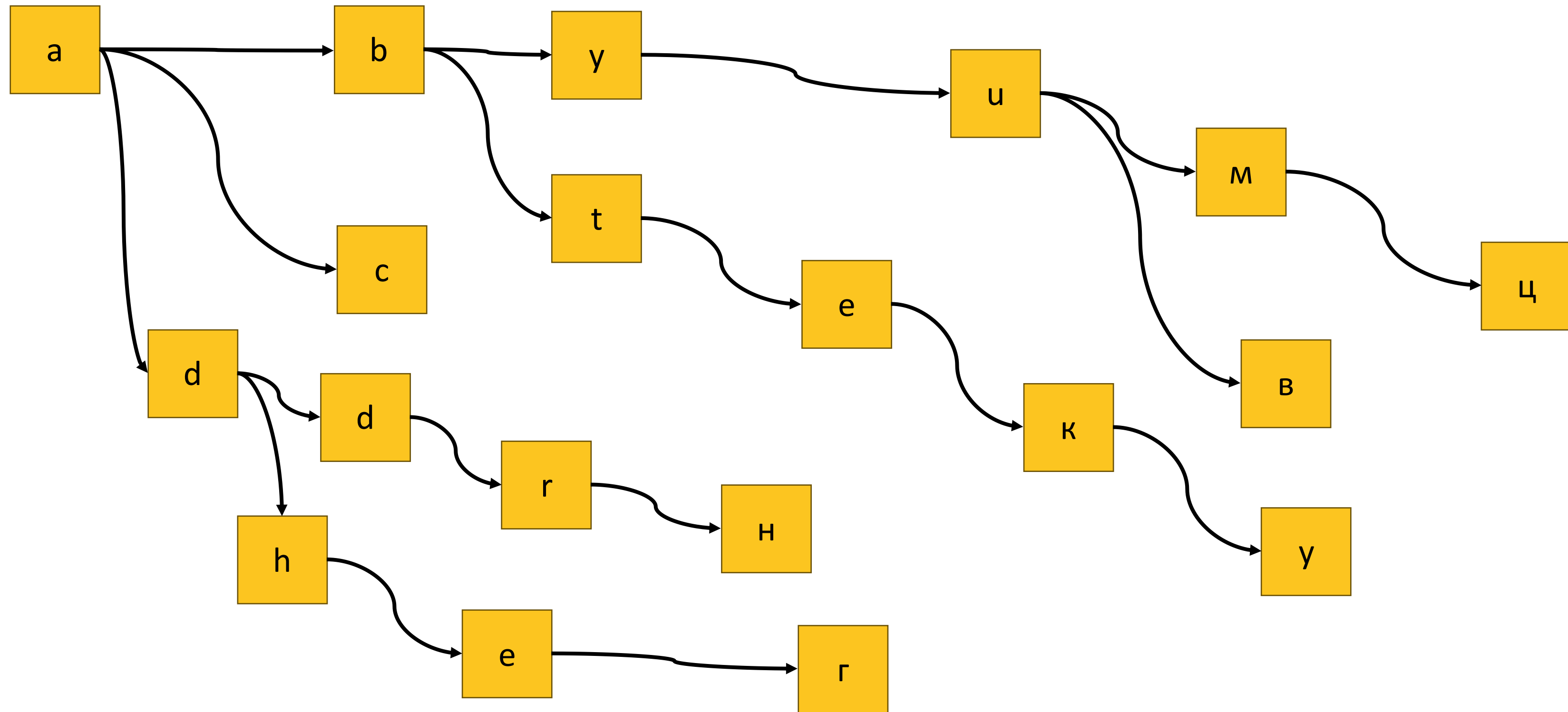
Текущая версия страницы пока **не проверялась** опытными участниками и может значительно отличаться от **версии, проверенной 16 июля 2025 года**; проверки требуют **2 правки**.

Авторегрессионная (AR-) модель (**англ.** *autoregressive model*) — модель **временных рядов**, в которой значения временного ряда в данный момент линейно зависят от предыдущих значений этого же ряда. Авторегрессионный процесс порядка p (AR(p)-процесс) определяется следующим образом

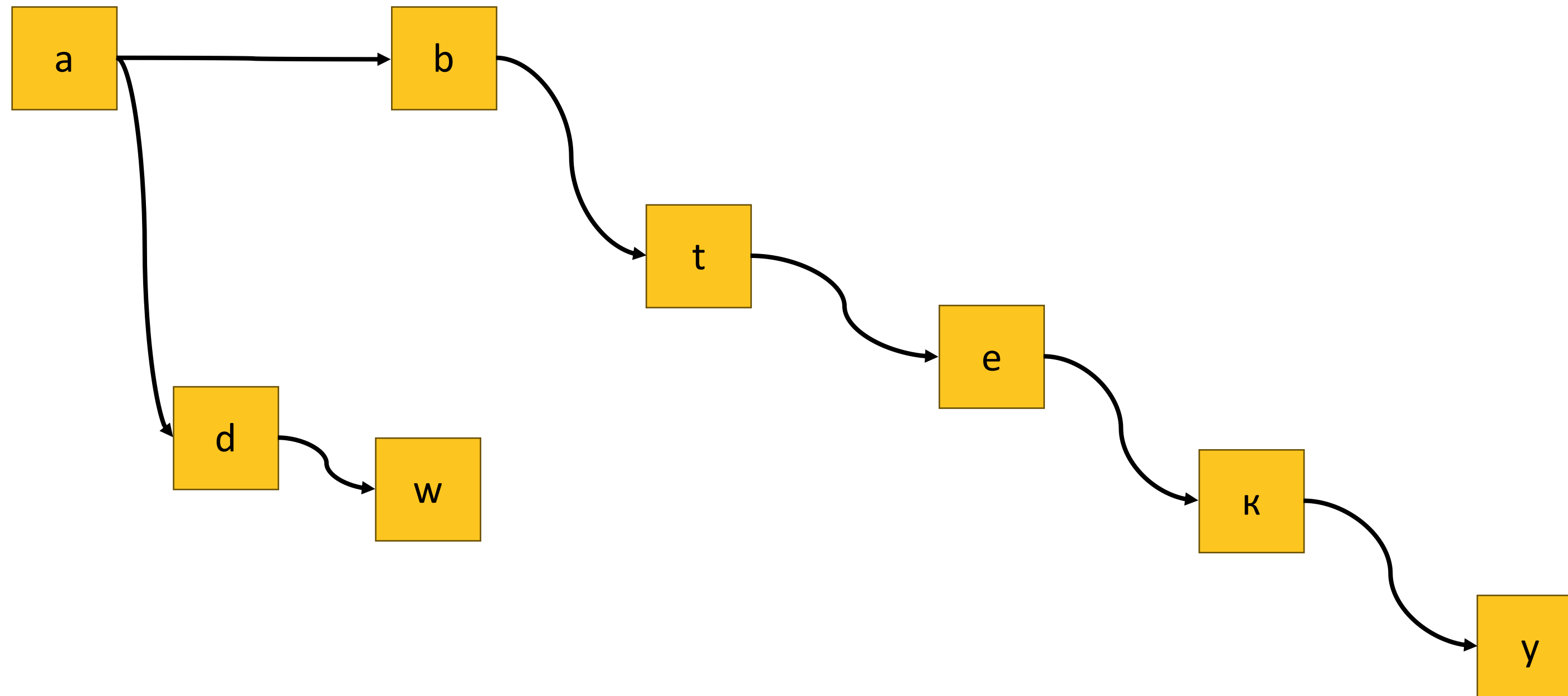
$$X_t = c + \sum_{i=1}^p a_i X_{t-i} + \varepsilon_t,$$

Разные пути пользователя

Есть много путей развития событий, но предпочтительнее самый длинный для нас



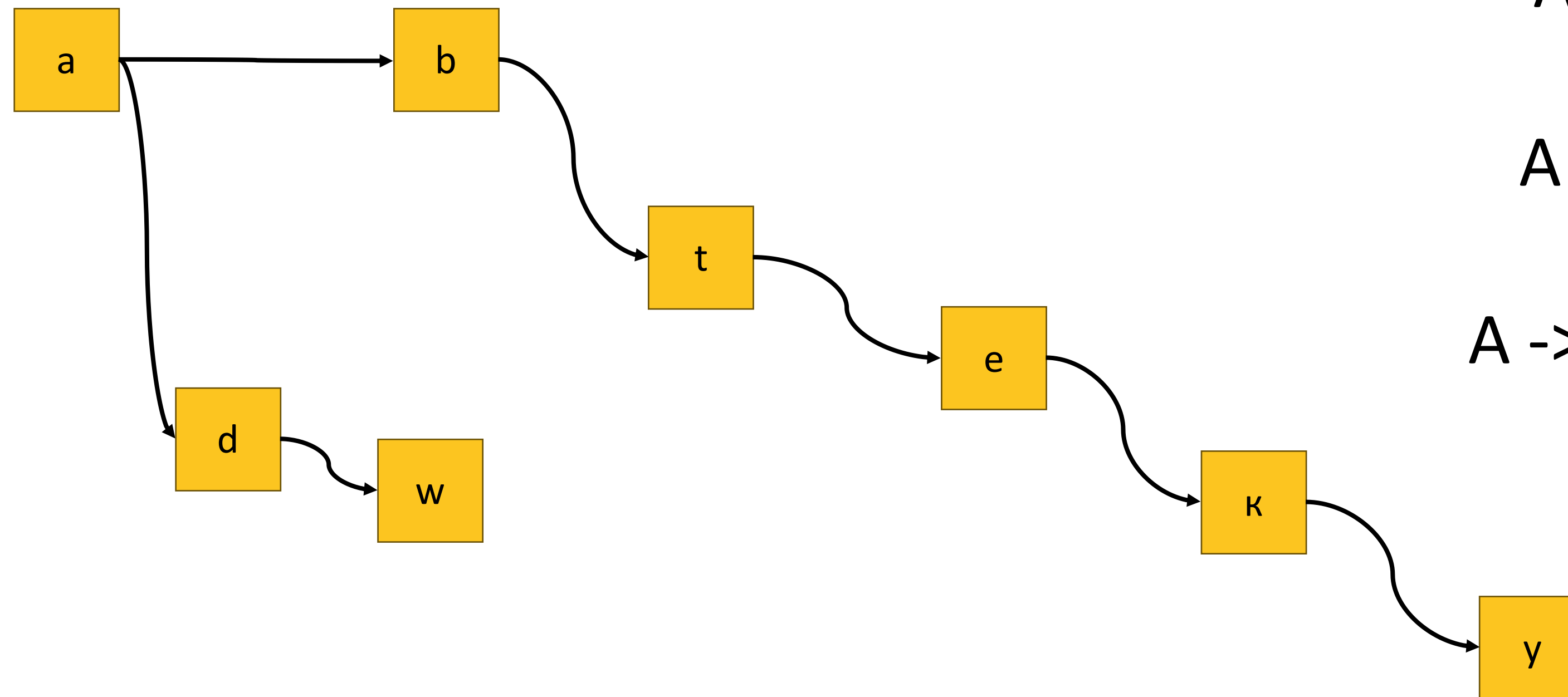
Как влияет recsys на вашу жизнь?



Показать вам айтем b или айтем d

- это повлияет на ваш дальнейший опыт, а возможно **и жизнь**

SASRec – это «языковая» модель языка ваших юзеров



$A \rightarrow b \rightarrow t$

$A \rightarrow d \rightarrow w$

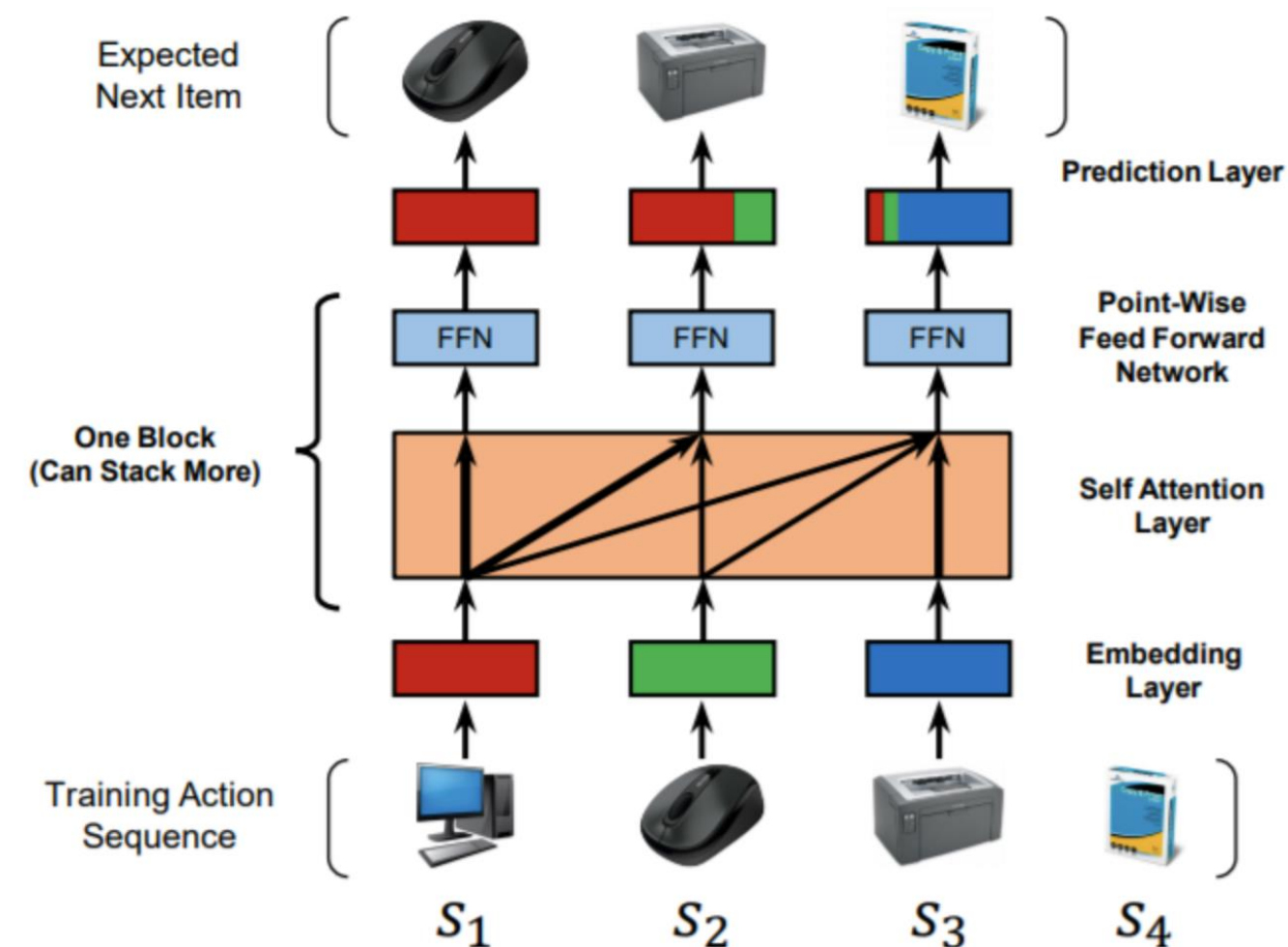
$A \rightarrow b \rightarrow t \rightarrow e$

Показать вам айтем b или айтем d

- это повлияет на ваш дальнейший опыт, а возможно **и жизнь**

SASRec – использует dot-product на инференсе

$\text{Score}(u, i)$ = вектор юзера u (из трансформера) * item embedding i с последнего слоя SASRec

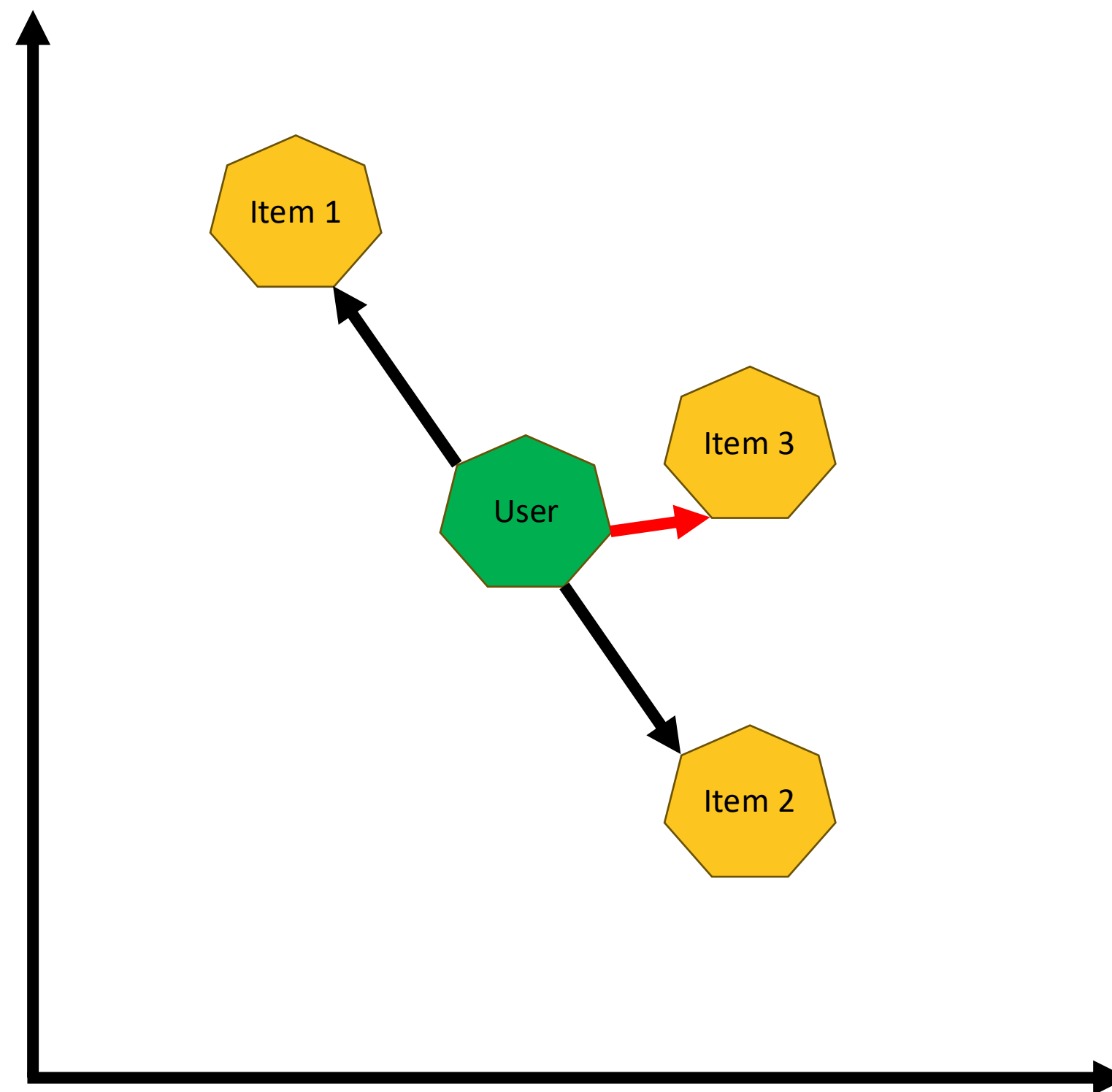


Тут можно использовать и ANN, ведь последний линейный слой – это есть матрица item embeddings.

Figure 1: A simplified diagram showing the training process of SASRec. At each time step, the model considers all previous items, and uses attention to ‘focus on’ items relevant to the next action.

Проблемы dot-product

1) Тяжело сделать непротиворечивые скоры user-item: loss на обучении по восстановлению user-item interactions $\neq 0$



User оценил item 1 и item 2. Вектор User'а находится оптимально. Но item3 ему не нравится, где тогда разместить вектор user'а?



Проблемы dot-product

2) Почти одинаковые items должны иметь разные скоры:

- допустим у вас есть два молока 1.3% и 2.3% жирности. Контентные вектора у них почти одинаковы, однако коллаборативные должны быть разными. Хорошо это или плохо

$S(u, \text{молоко } 1.3\%) = 0.85$, но $s(u, \text{молоко } 2.3\%) = 0.1$, поэтому вектора коллаборативные очень разные должны быть. Это противоречит контентной близости!



Проблемы dot-product

3) Айтемы надо «предсказывать целиком». В NLP моделях слова раскладываются на токены.

Токен – это сущность, которую обрабатывает модель. Токенизатор – функция, которая разбивает на токены текст

Землетрясение (один айтем).

Варианты токенизации:

1) з-е-м-л-я-т-р-я-с-е-н-и-е (побуквенно)

2) земля-тряс-ение (всего три токена). Причем каждый из трех токенов будет иметь смысл (земля, тряс, ение).

Причем если сказать «землетряс», скорее всего захочется добавить «ение».

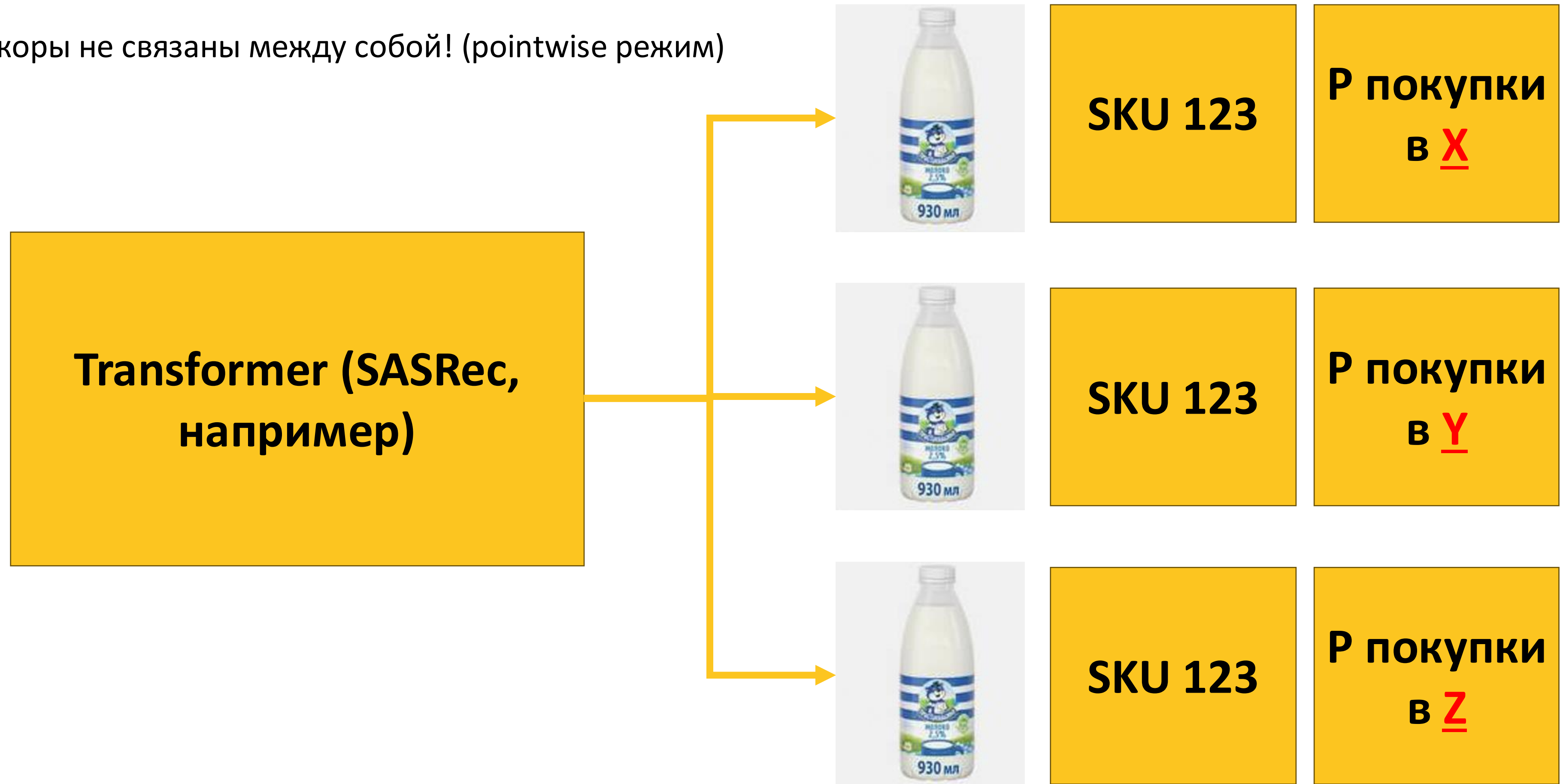
3) Землетрясение – один токен = одно слово. Проблема out-of-vocabulary (не факт, что вы знаете все слова, которые встречаются)

SASRec же предсказывает «айтем» землетрясение целиком (третий вариант)!

Редкие айтемы сложно выучить, на каждый надо много данных!

Проблемы dot-product

4) Скоры не связаны между собой! (pointwise режим)



Проблемы dot-product

Вот такая выдача:



Проблемы dot-product

Почему это скорее плохо?



Да

Нет

**Добавить в
корзину/перейти в
карточку товара**

**Не надо мне его
показывать еще
раз**

Проблемы dot-product

Почему это скорее плохо?

Да

**Добавить в
корзину/перейти в
карточку товара**

Нет

**Не надо мне его
показывать еще
раз**



Проблемы dot-product

Почему это скорее плохо?

Да

Нет

Добавить в
корзину/перейти в
карточку товара

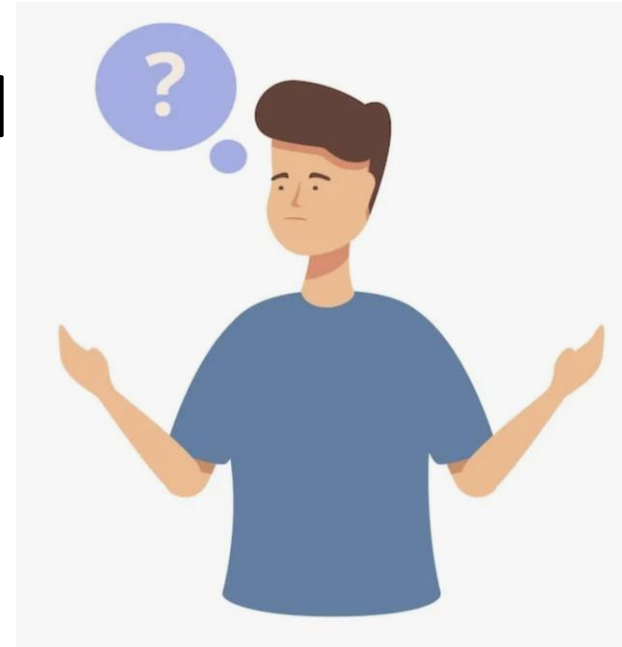
Не надо мне его
показывать еще
раз



Проблемы dot-product

5) dot-product почти не работает с контентными векторами

Пусть у вас вектор юзера – от SASRec’а, но вектор item – контентный.



Пусть также у вас в каталоге 100 батонov, и юзер релевантен один из них.

Утверждение: вы либо рекомендуете 100 батонov сразу, либо 0. Нельзя порекомендовать только один на контентных векторах

Проблемы dot-product

- 1) Противоречия в размещении user-item в пространстве
- 2) Почти одинаковые items с точки зрения контента будут расположены далеко
- 3) Айтемы надо предсказывать целиком в SASRec, не по токенам
- 4) Скоры user-item считаются независимо и могут вызывать дубли в выдачах
- 5) Айтем нельзя представить только контентно

Также есть проблема больших матриц айтемов

Может не влезть в память!

Dataset	Num Items	GPU Memory (one-token-per-item mode)
MovieLens-1M	3,416	3.34 MB
MovieLens-20M	138,493	135.25 MB
Yelp	150,346	146.82 MB
Gowalla	1,280,969	1.22 GB
Amazon Books	5,264,307	5.02 GB
LastFM-1b	32,291,134	30.80 GB

Токенизируемся

Пусть у нас есть 1М айтемов, это 1М эмбеддингов.

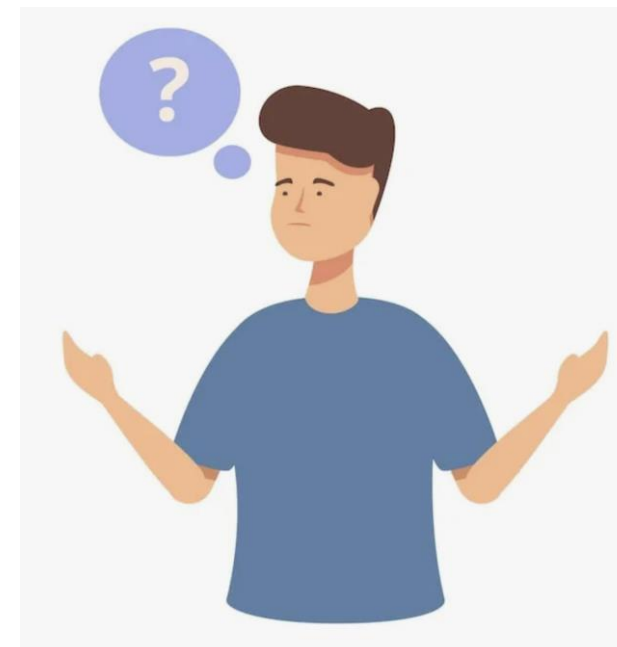
Давайте представим каждый айтем как набор из трех токенов, каждый из которых занимает 128 вариантов.

Тогда любой айтем представим набором токенов $X - Y - Z$

Всего вариантов у нас получается $128 * 128 * 128 = 2M$ вариантов можем покрыть.

То есть мы берем и для каждого `item_id` представляем какой-то набор $X - Y - Z$.

Сколько у нас всего будет эмбеддингов теперь?



Токенизируемся

Пусть у нас есть 1М айтеров, это 1М эмбеддингов.

Давайте представим каждый айтем как набор из трех токенов, каждый из которых занимает 128 вариантов.

Тогда любой айтем представим набором токенов $X - Y - Z$

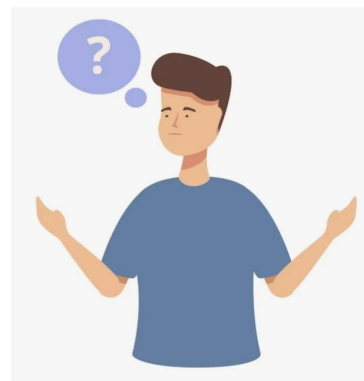
Всего вариантов у нас получается $128 * 128 * 128 = 2M$ вариантов можем покрыть.

То есть мы берем и для каждого item_id представляем какой-то набор $X - Y - Z$.

Сколько у нас всего будет эмбеддингов теперь? **128+128+128**

То есть вместо 1М эмбеддингов мы теперь храним 384 эмбеддинга!

Есть ли минусы?



Токенизируемся

Пусть у нас есть 1М айтемов, это 1М эмбеддингов.

Давайте представим каждый айтем как набор из трех токенов, каждый из которых занимает 128 вариантов.

Тогда любой айтем представим набором токенов $X - Y - Z$

Всего вариантов у нас получается $128 * 128 * 128 = 2M$ вариантов можем покрыть.

То есть мы берем и для каждого item_id представляем какой-то набор $X - Y - Z$.

Сколько у нас всего будет эмбеддингов теперь? **128+128+128** (но можно и вообще только 128)

То есть вместо 1М эмбеддингов мы теперь храним 384 эмбеддинга!

Есть ли минусы? Да, теперь мы потратим $3X$ токенов на представление той же инфы

Токенизируемся

Вот у нас есть последовательность юзера из item-id: 234 235 26 270 56 409

После маппинга мы можем сделать 1-1-1 2-1-2 2-1-3 3-1-4 4-1-5 5-1-6. Это то же самое, если у нас есть маппинг 1 в 1 из оригинального айди в составной айдишник.

234 -> 1-1-1

245 -> 2-2-1

26 -> 2-1-3 и тд..

Можем ли мы теперь выучить что-то вроде SASRec на новой последовательности?



Токенизируемся

Вот у нас есть последовательность юзера из item-id: 234 235 26 270 56 409

После маппинга мы можем сделать 1-1-1 2-1-2 2-1-3 3-1-4 4-1-5 5-1-6. Это то же самое, если у нас есть маппинг 1 в 1 из оригинального айди в составной айдишник.

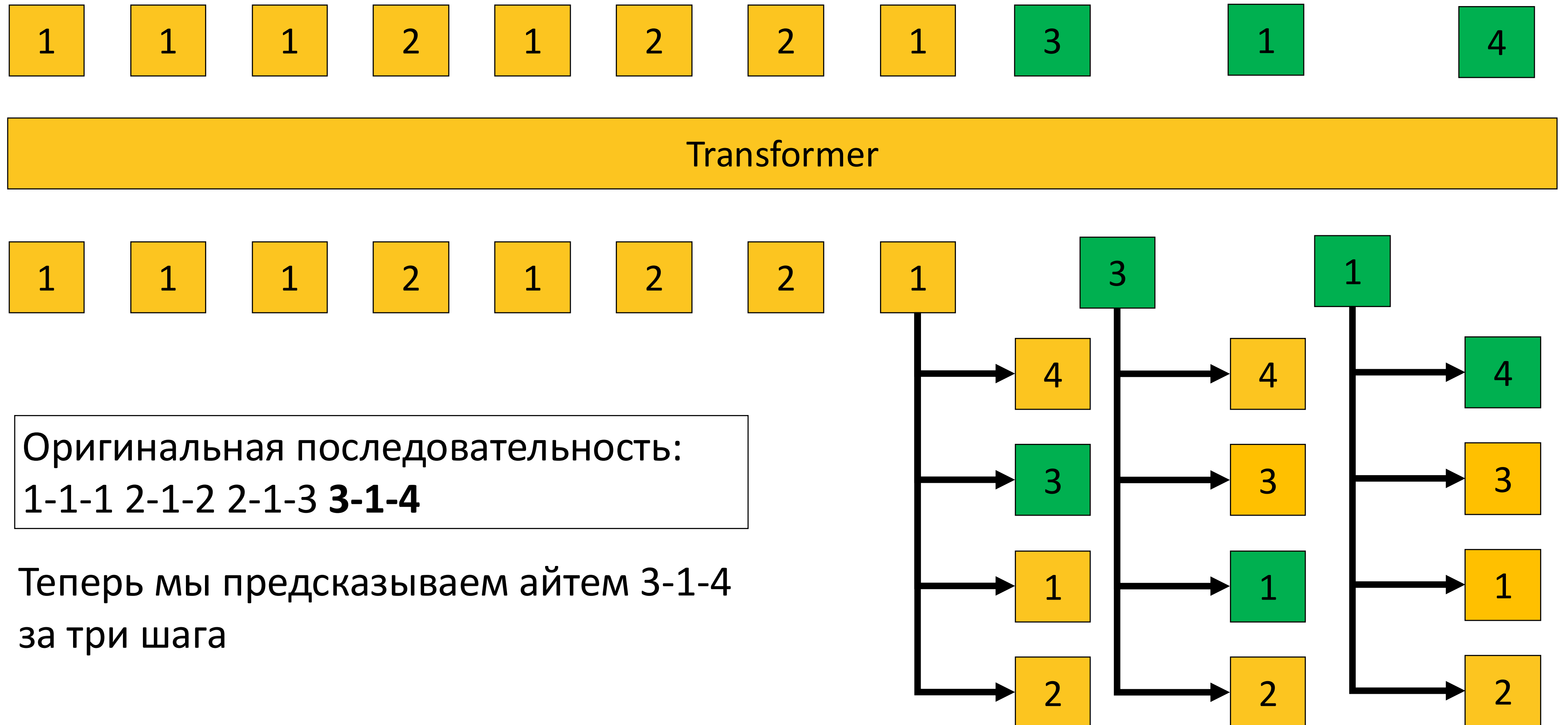
234 -> 1-1-1

245 -> 2-2-1

26 -> 2-1-3 и тд..

Можем ли мы теперь выучить что-то вроде SASRec на новой последовательности? **Да!**

Решаем задачу next token prediction



Минусы генеративной генерации



Минусы генеративной генерации



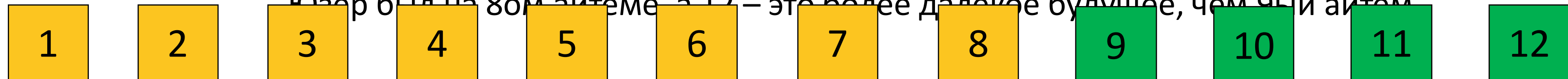
1) Мы тратим больше времени, теперь на каждый айтем надо три прогона трансформера.

На 10 айтемов надо x30 прогонов

2) Трансформер может сгенерировать несуществующий айтем. У нас всего 1M айтемов, но вариантов 2M. Половина всевозможных комбинаций не существует!

3) Если мы сгенерируем 10 следующих айтемов, мы уйдем далеко от текущего стейта.

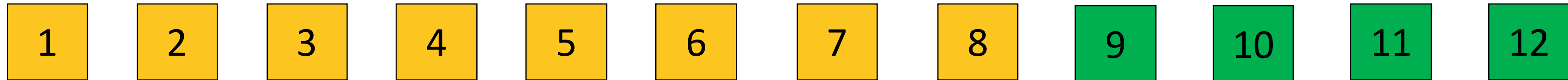
Юзер был на 8ом айтеме, а 12 – это более далекое будущее, чем 9ый айтем



Решаем ли минусы dot-product?

- 1) Противоречия в размещении user-item в пространстве. **Скорее да, здесь такое сложно придумать**
- 2) Почти одинаковые items с точки зрения контента будут расположены далеко. **Не факт**
- 3) Айтемы надо предсказывать целиком в SASRec, не по токенам. **Решаем напрямую**
- 4) Скоры user-item считаются независимо и могут вызывать дубли в выдачах. **Решается, так как каждый айтем в рекомендациях зависит от уже сгенерированных в выдаче**
- 5) Айтем нельзя представить только контентно. **Можно, про это дальше**

Top-K vs Next-K generation



Transformer

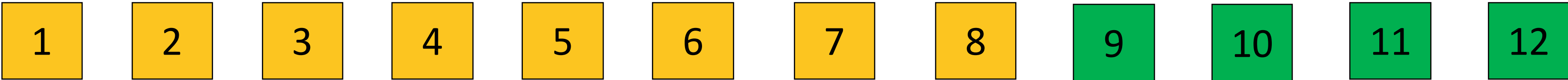
8ой айтем – самый последний в интеракциях, мы можем:

1) top-k генерация – сразу после 8ого айтема набрать самые вероятные айтемы:

Если айтем – набор токенов, можно сделать beam search/sampling, то есть набрать K самых релевантных айтемов сразу после 8

2) next-k генерация: мы рекомендуем айтем 9, ставим его в сиквенс, и идем дальше. Мы отдаляемся от последнего действия юзера, но смотрим дальше в его историю

Top-K vs Next-K generation



Transformer

	Плюсы	Минусы
Top-K	Не накапливаем ошибку, не придумываем ничего	Не можем выучивать долгосрочные стратегии рекомендаций
Next-K	«ИИ предсказал беременность» или заглядываем в будущее клиента	Дольше времени занимает генерация Метрики ниже

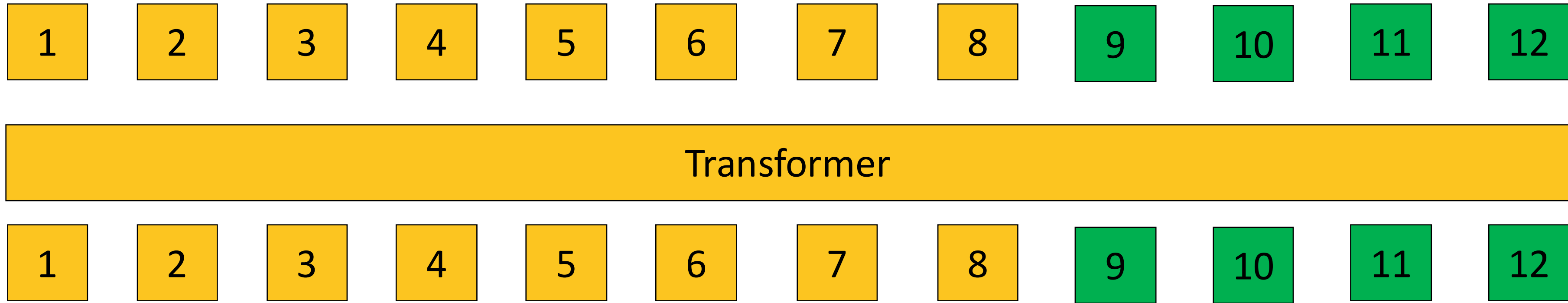
Gen-IR@SIGIR2023, July 27, 2023, Taipei, Taiwan

Aleksandr V. Petrov and Craig Macdonald

Table 3: Model performances on the MovieLens-1M dataset. Bold values indicate the best results, and underlined values indicate the second best results. SASRec and BERT4Rec results are copied from the reproducibility paper [18].

Model name	Generation Strategy	Architecture	Training Task	Loss	Recall@10	NDCG@10
BERT4Rec	TopK	Encoder	MLM (Item Masking)	Cross Entropy (Softmax Loss)	0.282	0.152
GPTRec-TopK	TopK	Decoder	LM (Sequence Shifting)	Cross Entropy (Softmax Loss)	<u>0.254</u>	<u>0.146</u>
SASRec	TopK	Decoder	LM (Sequence Shifting)	Binary Cross-Entropy	0.199	0.108
GPTRec-NextK	NextK	Decoder	LM (Sequence Shifting)	Cross Entropy (Softmax Loss)	0.157	0.105

Инструментарий из LLM про Pretraining+SFT + RLHF



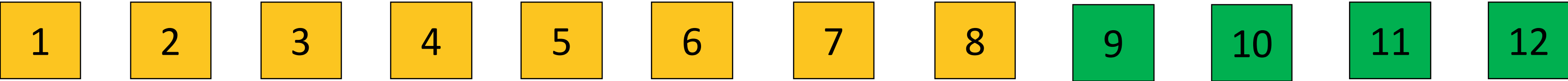
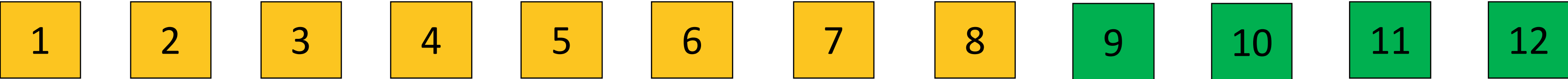
Сначала вам надо сделать базовую предобученную модельку уметь говорить на языке юзеров.

Эта стадия называется pretraining (предобучение)

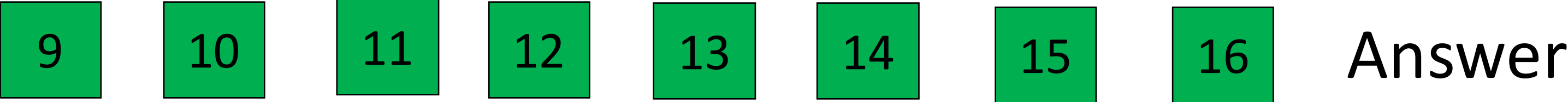
Такая готовая моделька умеет продолжать последовательности (как SASRec), но не факт, что хорошо решает наши задачи.

По факту, SASRec – это и есть предобученная моделька на ваших интеракциях

Инструментарий из LLM про Pretraining+SFT + RLHF

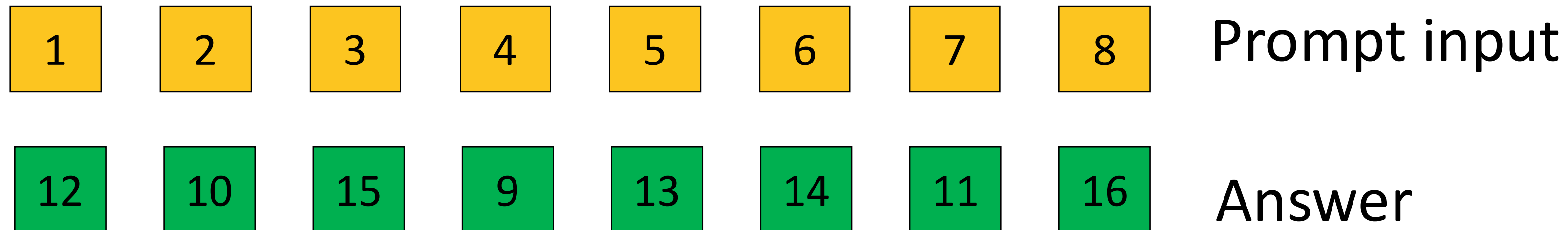


Поверх базовой предобученной модели (pretraining), можно сделать Supervised Fine-Tuning:



Как собрать Answer – на ваше усмотрение, но здесь (в отличие от Pretraining на NTP) вы можете собирать самые релевантные айтемы на момент времени айтема 8 (например, это айтемы 13-16)

Инструментарий из LLM про SFT + RLHF

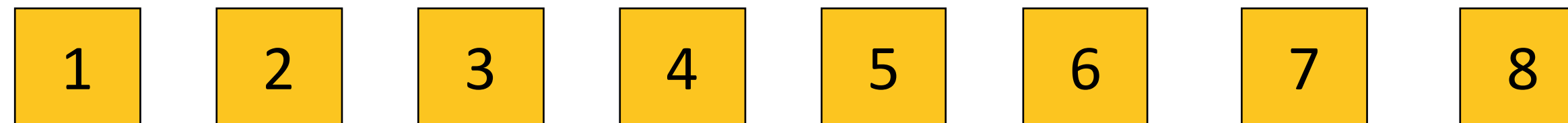


Через SFT вы можете next-k генерацию по факту превратить в top-k!

Инструментарий из RL для LLM



Вы передали:



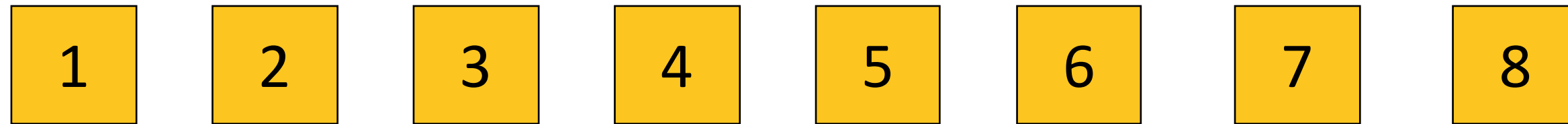
Модель сгенерировала:



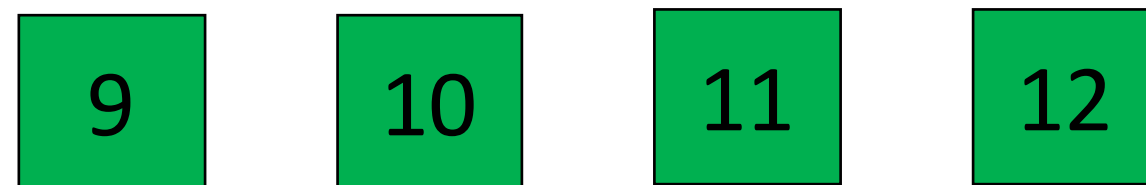
Это уже не next-items реального юзера, это какая-то абстрактная генерация рекомендаций ему в момент времени последний

Инструментарий из RL для LLM

Вы передали:



Модель сгенерировала:



Что вы хотите от своего списка?

- 1) Может быть хорошей метрики NDCG?
- 2) Может быть отсутствие айтеров из одной категории?
- 3) Может быть айтеров, которых юзер еще не видел?

Вы можете использовать RL для оптимизации политики рекомендаций, как и в LLM!

Инструментарий из RL для LLM

Вы передали:

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8

Модель сгенерировала:

- 9101112
- 124249321
- 1211

Функция награды:

Релевантность	Длина списка	Разнообразие	Награда
0.3	4	0.2	4.5
0.2	5	0.1	5.3
0.1	3	0	3.1

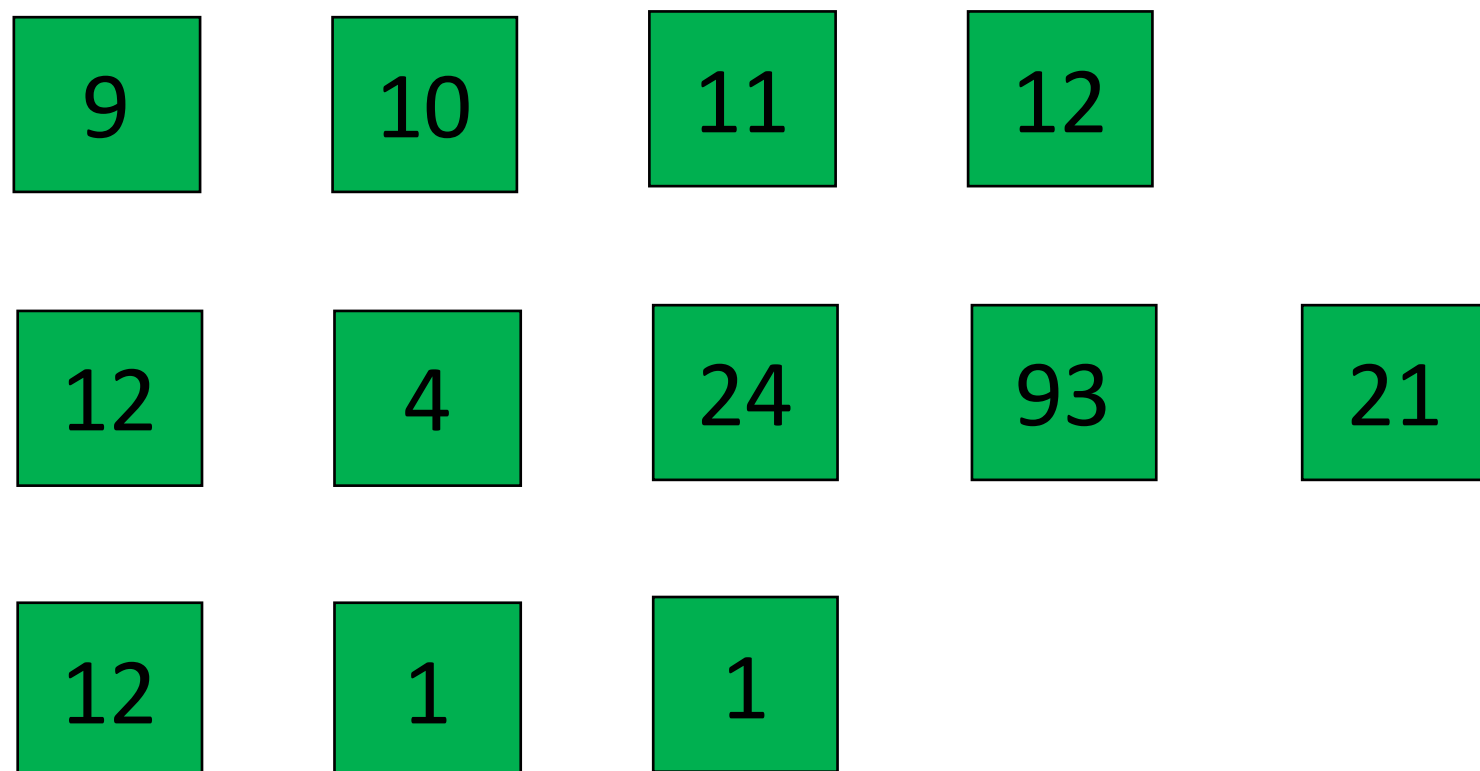
RL (GRPO/DPO/PPO) будут двигать генерацию в сторону второго варианта

Инструментарий из RL для LLM

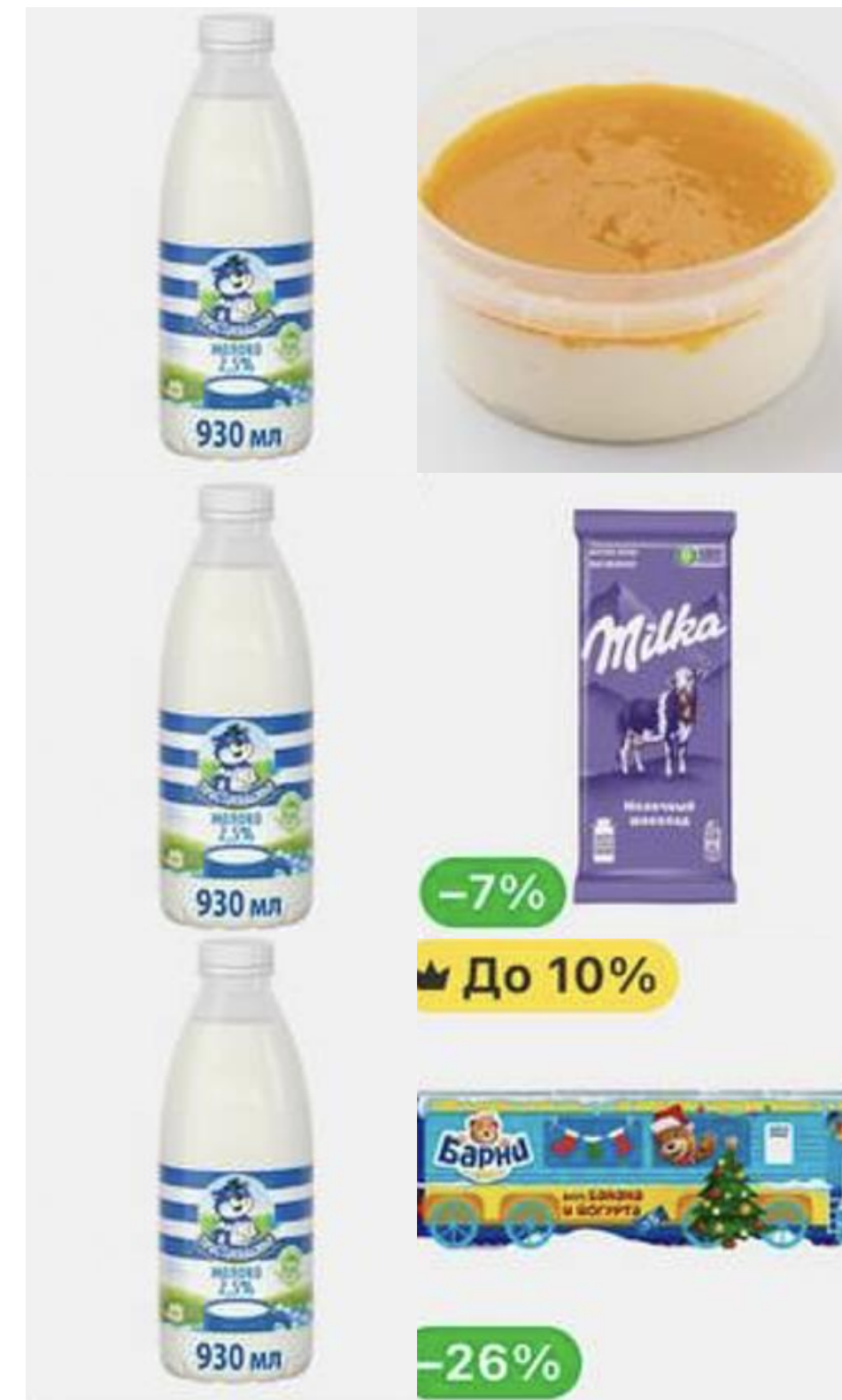
Вы передали:



Модель сгенерировала:



Функция награды:



Такие ленты мы запрещаем генерировать через функцию награды

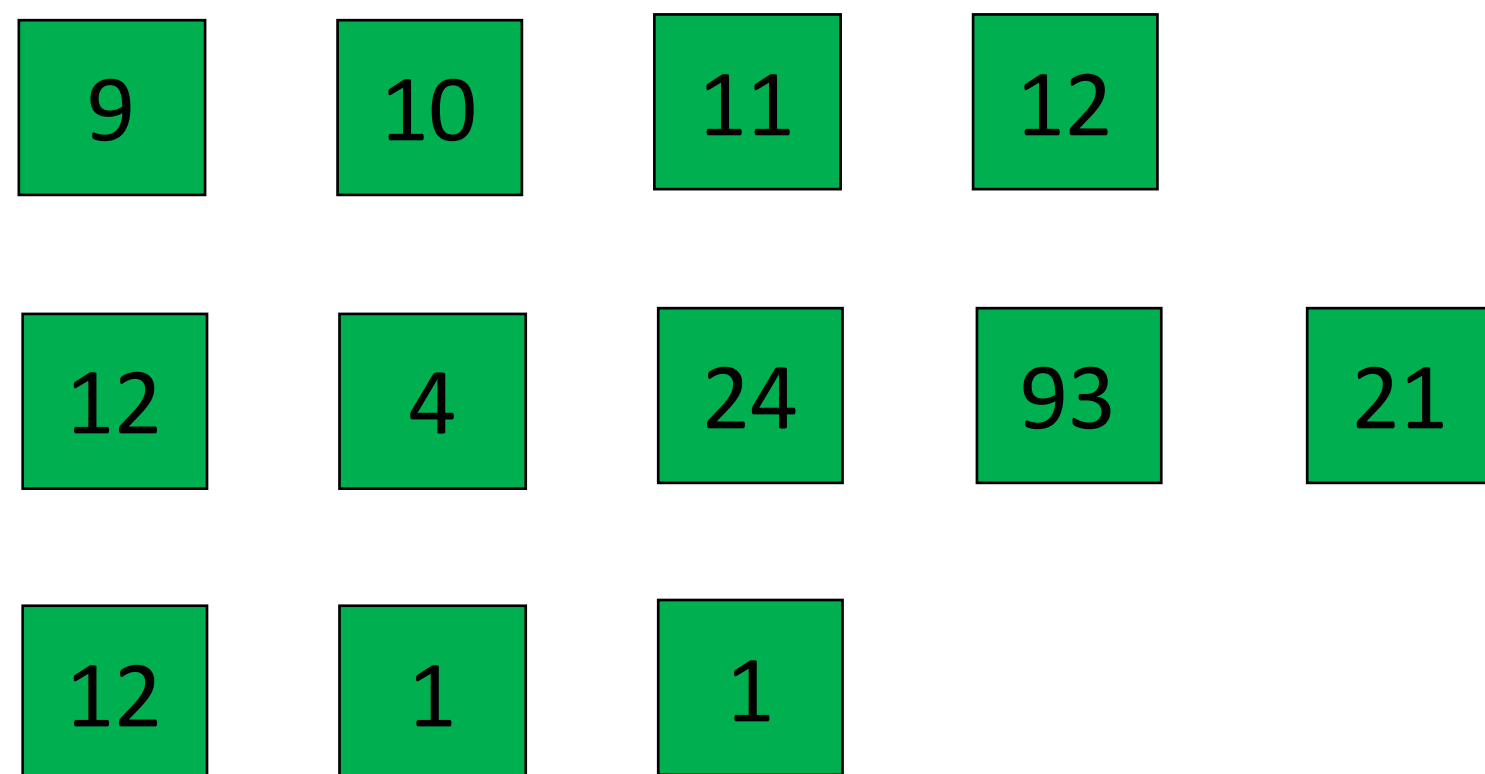
Функция награды может быть сложнее (reward modeling)



Вы передали:



Модель сгенерировала:



Давайте предскажем, насколько юзер далеко проскролит?

Это просто трансформер, который предсказывает 1/0 (reward model) на основе

- юзер действий
- логов показов рекомендаций
- фактов того, что юзер закрыл реки, либо листал дальше (бинарная классификация)

Функция награды может быть сложнее (reward modeling)



Вы передали:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Модель сгенерировала:

9	10	11	12
12	4	24	93
12	1	1	

Вероятности «отвала»

0.3	0.6	0.1	0.3
0.1	0.4	0.6	0.2
0.1	0.2	0.3	

Есть последовательность испытаний $i = 1, 2, \dots$

Для каждого испытания известна вероятность фейла:

$$p_i = P(\text{fail на } i \mid \text{дошли до } i)$$

Обозначим случайную величину:

$$T = \text{номер первого фейла}$$

Нужно найти:

$$\mathbb{E}[T]$$

1 Общий случай (вероятности разные)

Вероятность того, что первый фейл произойдёт на шаге i :

$$P(T = i) = \left(\prod_{k=1}^{i-1} (1 - p_k) \right) p_i$$

(надо не зафейлиться на первых $i - 1$ шагах и зафейлиться на i).

Тогда матожидание:

$$\mathbb{E}[T] = \sum_{i=1}^{\infty} i \left(\prod_{k=1}^{i-1} (1 - p_k) \right) p_i$$

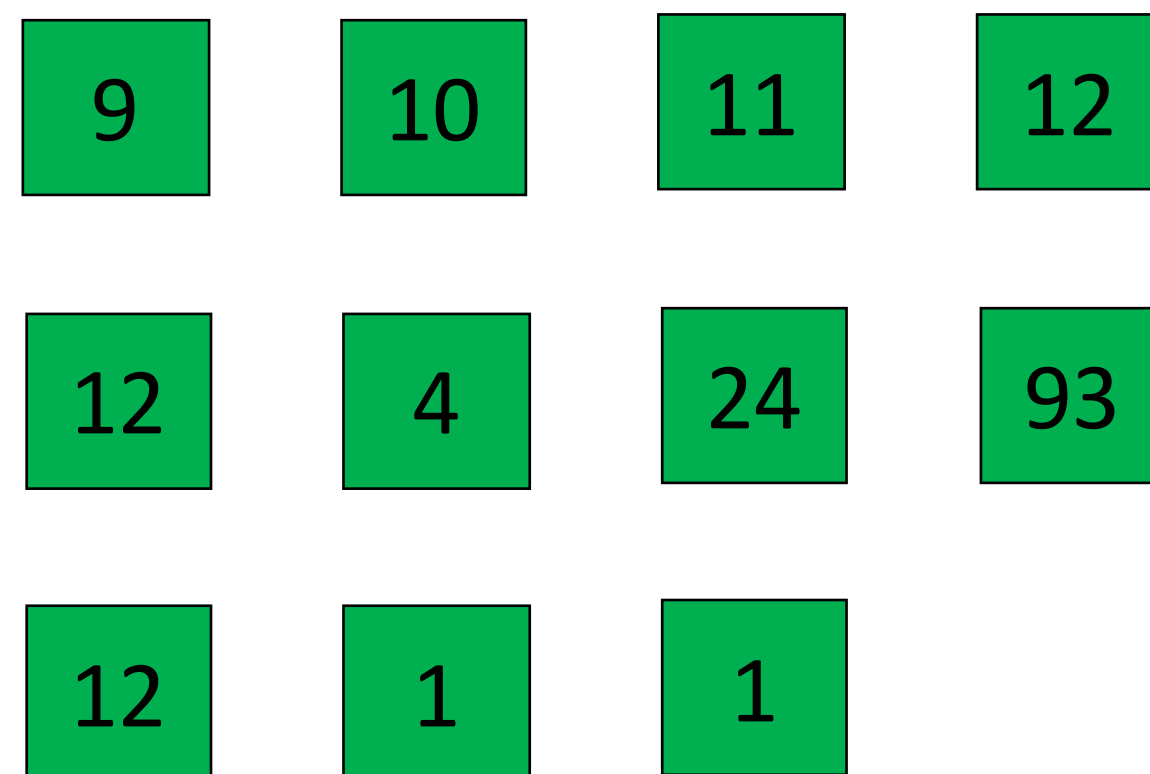
Функция награды может быть сложнее (reward modeling)



Вы передали:



Модель сгенерировала:



Модель на retention, через сколько вернется юзер?
Это обычный supervised learning (У вас есть разметка)

2 часа

Это предсказание supervised reward модели, что будет, если дать такой список рекомендаций юзеру.

1 день

У нее есть ошибки.

Не вернется

Но мы хотим показать такой список, чтобы юзер вернулся через максимально быстрое время

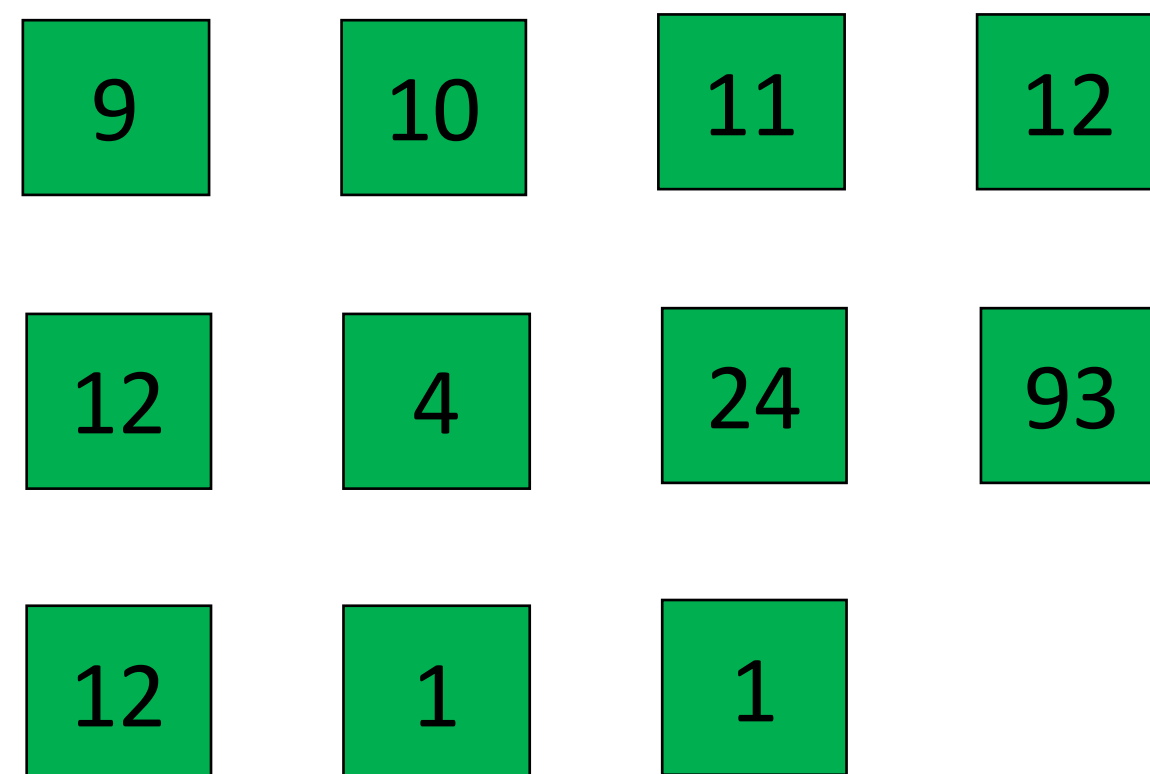
Функция награды может быть сложнее (reward modeling)



Вы передали:



Модель сгенерировала:



Модель на retention, через сколько вернется юзер?
Это обычный supervised learning (У вас есть разметка)

2 часа

Это предсказание supervised reward модели, что будет, если дать такой список рекомендаций юзеру.

1 день

У нее есть ошибки.

Не вернется

Но мы хотим показать такой список, чтобы юзер вернулся через максимально быстрое время

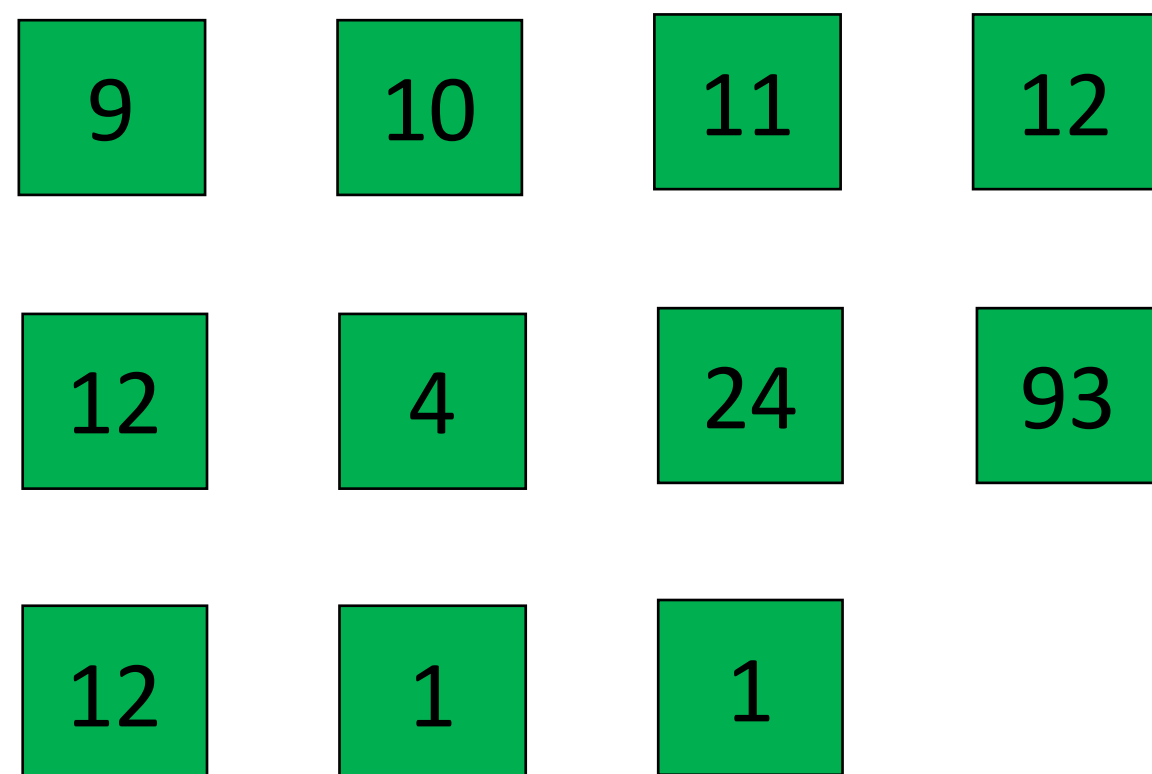
Функция награды может быть сложнее (reward modeling)



Вы передали:



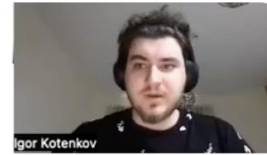
Модель сгенерировала:



Можете попросить оценить LLM
каждый список рекомендаций с
учетом прошлой истории юзера и
попросить оценить
рекомендации от 0 до 10

Доп. Материал про RL

RLHF Intro: from Zero to Aligned Intelligent Systems



Igor Kotenkov



- 1) Получится ли у вас построить хорошую reward model или она будет ошибаться?
- 2) Есть ли у вас вообще данные, чтобы сделать то, что вы хотите?
- 3) Вы можете хакнуть reward model, но не добиться результата.

Вопрос – зачем нам нужна модель генерации рекомендаций, если мы можем по reward model выдавать хорошие рекомендации?

Еще один кейс для RL

Пусть у вас есть много бизнес правил для формирования списка рекомендаций.

Например, у вас есть сервис подписки, в рамках которого доступно X опций. Ваша задача – составить список длины K , который должен строго удовлетворять правилам:

- 1) А и Б не должны быть в одном списке
- 2) В и Г можно выдавать только вместе
- 3) Если есть А, и Б, тогда не должно быть В и Г
- 4) Если есть А, Б и В, надо обязательно дать либо Г, либо Д, либо С
- 5) Надо обязательно дать либо В, либо Г
- 6) Должно быть не больше 3 гласных

Причем эти бизнес правила могут зависеть от клиента!

То есть если у клиента есть W , то правило такое...

9

10

11

12

Вы можете написать функцию награды, которая будет удовлетворять всем правилам, но причем еще максимизировать счастье пользователя.

Этот кейс – вы умеете проверять решение, но не умеете его генерировать. Это как с доказательствами теорем в математики.

Люди могли проверить корректность доказательства теории Пуанкаре, но не могли ее доказать.

Через RL такие решения в рекомендациях найти возможно

Вернемся к вопросу токенизации

Вот у нас есть последовательность юзера из item-id: 234 235 26 270 56 409

После маппинга мы можем сделать 1-1-1 2-1-2 2-1-3 3-1-4 4-1-5 5-1-6. Это то же самое, если у нас есть маппинг 1 в 1 из оригинального айди в составной айдишник.

234 -> 1-1-1

245 -> 2-2-1

26 -> 2-4-3 и тд..

Проблема: если 26 и 234 айтемы похожи, у них совсем разные айдишники.

Семантическая токенизация

Пусть пользователь 1 купил: молоко, яйца, дрожжи, изюм, чай.

А пользователь 2 купил: кефир, яйца, дрожжи, курагу, кофе.

Возможно, оба пользователя хотят испечь пироги с изюмом или курагой и съесть их с чаем и кофе.

Если мы закодируем сиквенсы как 1-1-1, 2-2-2, 3-3-3, 4-4-4, 5-5-5 и
6-6-6 2-2-2 3-3-3 9-9-9 10-10-10

Модель вообще не поймет, что эти паттерны чем-то похожи

Семантическая токенизация

Товар	Рандомная токенизация	Семантическая токенизация
Молоко	1-1-1	1-1-1
Яйца	2-2-2	2-2-2
Дрожжи	3-3-3	3-3-3
Изюм	4-4-4	4-4-4
Чай	5-5-5	5-5-5
Кефир	6-6-6	1-1-2
Курага	7-7-7	4-4-5
Кофе	8-8-8	5-5-6

1-1-1, 2-2-2, 3-3-3, 4-4-4, 5-5-5 и 6-6-6, 2-2-2, 3-3-3, 9-9-9, 10-10-10	1-1-1, 2-2-2, 3-3-3, 4-4-4, 5-5-5 и 1-1-2, 2-2-2, 3-3-3, 4-4-5, 5-5-6
---	--

Теперь для модели эти истории более связанны, и она более уверенно может порекомендовать муку для теста к пирога.

Семантическая токенизация

Товар	Рандомная токенизация	Семантическая токенизация
Молоко	1-1-1	1-1-1
Яйца	2-2-2	2-2-2
Дрожжи	3-3-3	3-3-3
Изюм	4-4-4	4-4-4
Чай	5-5-5	5-5-5
Кефир	6-6-6	1-1-2
Курага	7-7-7	4-4-5
Кофе	8-8-8	5-5-6

Как можно сделать такое несложными методами?



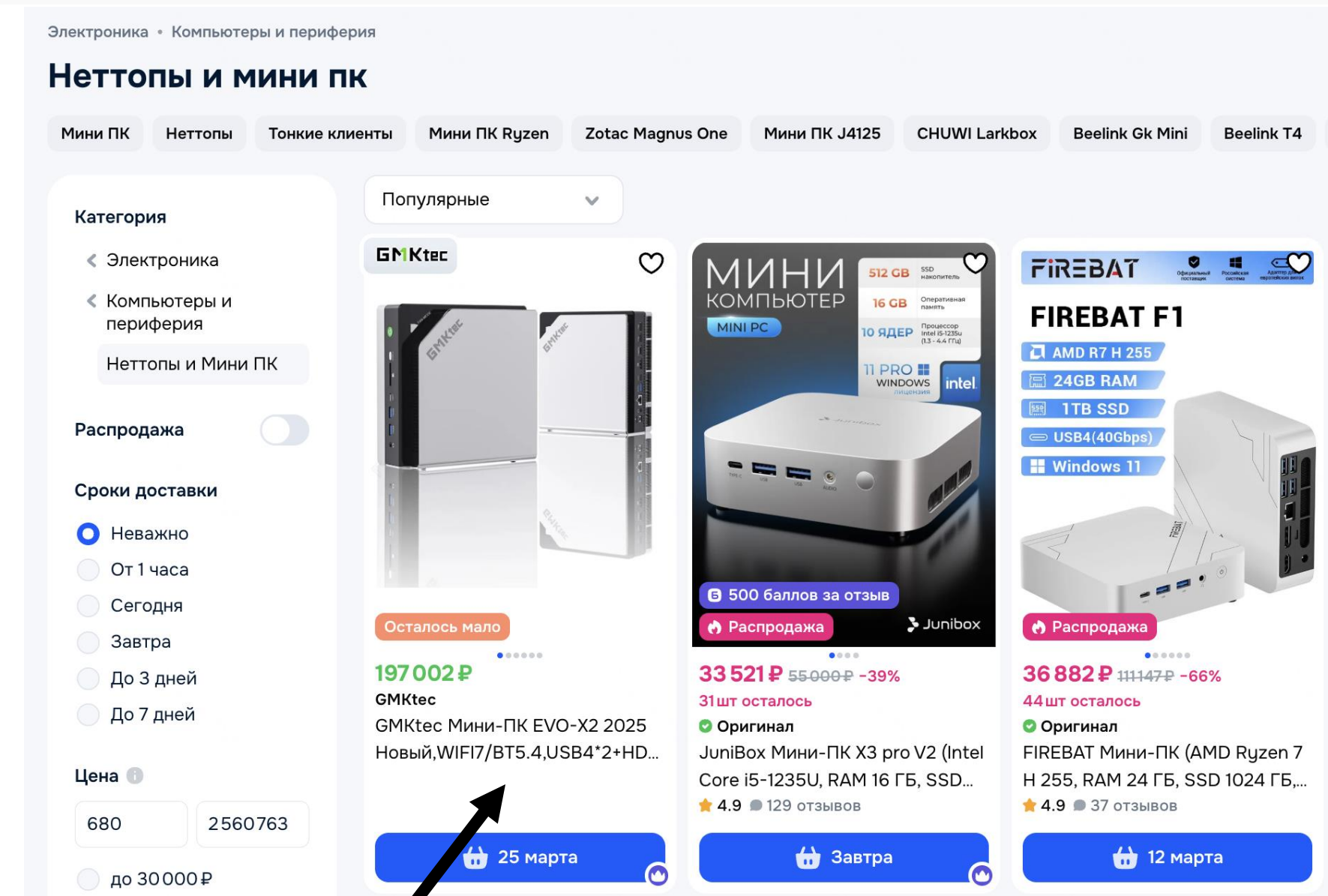
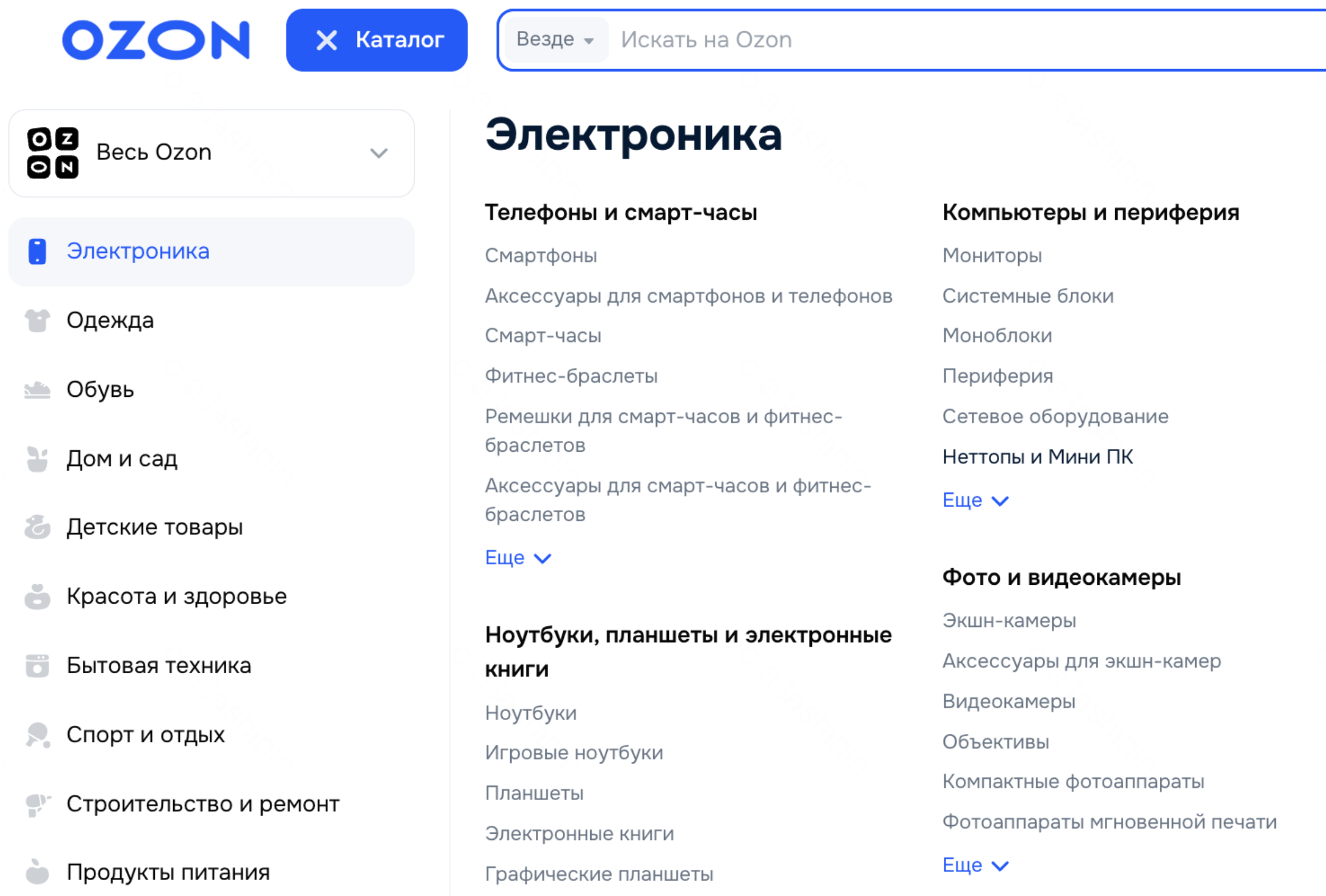
Семантическая токенизация

Товар	Рандомная токенизация	Семантическая токенизация
Молоко	1-1-1	1-1-1
Яйца	2-2-2	2-2-2
Дрожжи	3-3-3	3-3-3
Изюм	4-4-4	4-4-4
Чай	5-5-5	5-5-5
Кефир	6-6-6	1-1-2
Курага	7-7-7	4-4-5
Кофе	8-8-8	5-5-6

Как можно сделать такое несложными методами?

Можно использовать готовую категоризацию

Семантическая токенизация



Id товара:

1-2-3-0

1 – Электроника

2 – Компьютеры и периферия

3 – Неттопы и Мини ПК

0 - айді внутри этой категории

Можно учиться на полных айди, или только на семантике

Вариант №1 (Полный). Семантическая часть + уникализирующая

1-2-3-0 -> 1-3-4-5 -> 2-4-5-2 -> 2-4-5-6 -> 2-4-1-2 -> 34-23-14-0 -> 2-35-2-3

Вариант №2 (Семантический). Только семантика

1-2-3 -> 1-3-4 -> 2-4-5 -> 2-4-5 -> 2-4-1 -> 34-23-14 -> 2-35-2

	Плюсы	Минусы
Семантика + уникализация	Декодирование айтемов явный Рекомендации сразу конкретных айтемов, а не только семантической части	Модель тратит вместимость на запоминание конкретных айдишников Привязанность к конкретному каталогу
Только семантика	Задача для модели легче – понять суть	Модель обладает меньшей информацией, что конкретно покупал юзер?

Вопрос декодинга кейса с семантикой

2-3-4-0

2-3-4-1

2-3-4-2

2-3-4-3



Осталось мало

197 002 Р

GMKtec

GMKtec Мини-ПК EVO-X2 2025
Новый,WIFI7/BT5.4,USB4*2+HD...

25 марта



500 баллов за отзыв

Распродажа

33 521 Р ~~55 000 Р~~ -39%

31 шт осталось

Оригинал

JuniBox Мини-ПК X3 pro V2 (Intel Core i5-1235U, RAM 16 ГБ, SSD...)

4.9 129 отзывов

Завтра



Распродажа

36 882 Р ~~111 147 Р~~ -66%

44 шт осталось

Оригинал

FIREBAT Мини-ПК (AMD Ryzen 7 H 255, RAM 24 ГБ, SSD 1024 ГБ,...)

4.9 37 отзывов

12 марта



Осталось мало

Цена что надо

88 085 Р

CHUWI Мини-ПК MiniPC AuBox AI (AMD Ryzen AI 9 365, RAM 32 ГБ, SSD 1024 ГБ, AMD Radeon...)

4.8 65 отзывов

8 марта

Модель рекомендует **2-3-4**
Что покажете юзеру?



Вопрос декодинга кейса с семантикой

2-3-4-0

2-3-4-1

2-3-4-2

2-3-4-3



Осталось мало

197 002 ₽

GMKtec
GMKtec Мини-ПК EVO-X2 2025
Новый, WIFI7/BT5.4, USB4*2+HD...

25 марта



500 баллов за отзыв

Распродажа

33 521 ₽ ~~55 000 ₽~~ -39%

31 шт осталось

Оригинал

JuniBox Мини-ПК X3 pro V2 (Intel Core i5-1235U, RAM 16 ГБ, SSD...)

★ 4.9 129 отзывов

Завтра



Распродажа

36 882 ₽ ~~111 147 ₽~~ -66%

44 шт осталось

Оригинал

FIREBAT Мини-ПК (AMD Ryzen 7 H 255, RAM 24 ГБ, SSD 1024 ГБ,...)

★ 4.9 37 отзывов

12 марта



Осталось мало

Цена что надо

88 085 ₽

CHUWI Мини-ПК MiniPC AuBox AI (AMD Ryzen AI 9 365, RAM 32 ГБ, SSD 1024 ГБ, AMD Radeon...)

★ 4.8 65 отзывов

8 марта

Модель рекомендует **2-3-4**

Что покажете юзеру?

- 1) топ-1 популярный внутри
- 2) Все айтемы из этой подкатегории
- 3) Рандом

Построение Semantic Ids по-умному

Recommender Systems with Generative Retrieval

Shashank Rajput* University of Wisconsin-Madison	Nikhil Mehta* Google DeepMind	Anima Singh Google DeepMind
Raghunandan Keshavan Google	Trung Vu Google	Lukasz Heldt Google
Yi Tay Google DeepMind	Vinh Q. Tran Google	Maciej Kula Google DeepMind
Ed H. Chi Google DeepMind	Maheswaran Sathiamoorthy Google DeepMind	

Recommender systems with generative retrieval

[S Rajput](#), [N Mehta](#), [A Singh](#)... - Advances in ..., 2023 - [proceedings.neurips.cc](#)

Modern recommender systems perform large-scale retrieval by embedding queries and item candidates in the same unified space, followed by approximate nearest neighbor search to select top candidates given a query embedding. In this paper, we propose a novel generative retrieval approach, where the retrieval model autoregressively decodes the identifiers of the target candidates. To that end, we create semantically meaningful tuple of codewords to serve as a Semantic ID for each item. Given Semantic IDs for items in a user ...

☆ Save 📄 Cite Cited by 581 Related articles All 6 versions 🔗

<https://github.com/EdoardoBotta/RQ-VAE-Recommender>

Построение Semantic Ids по-умному

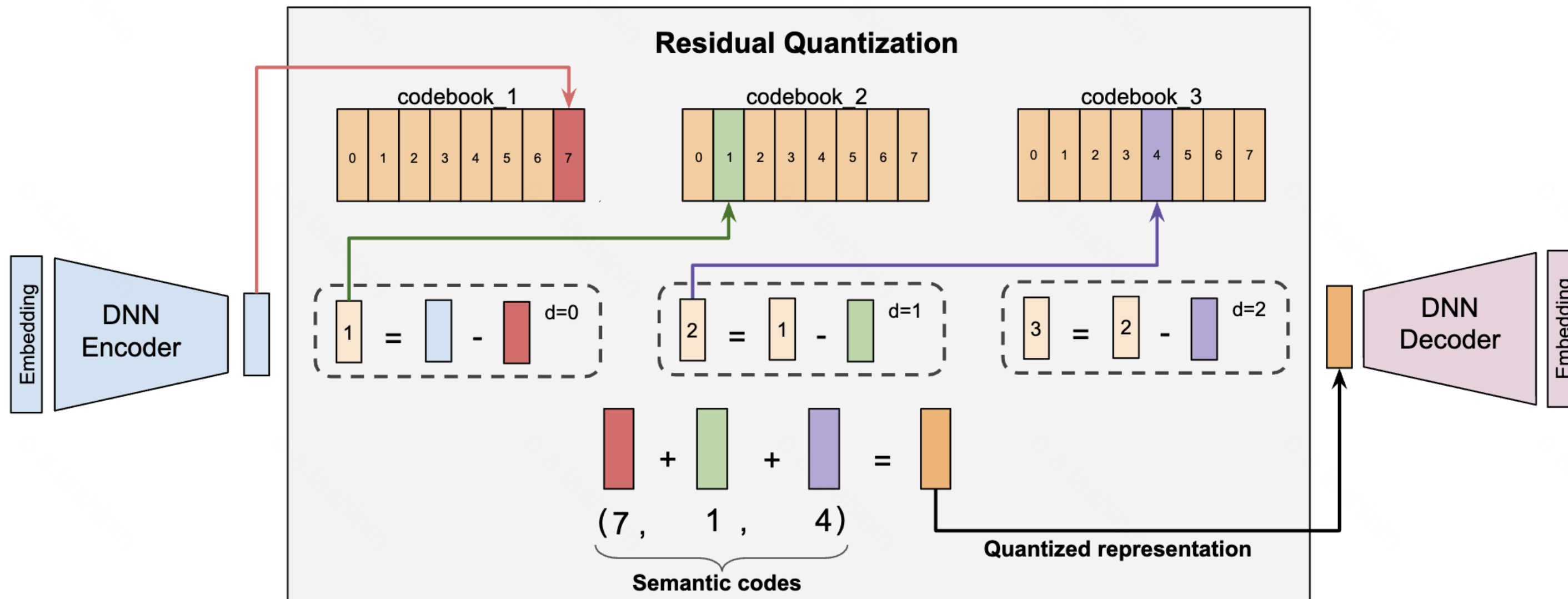


Figure 3: RQ-VAE: In the figure, the vector output by the DNN Encoder, say $\mathbf{r}_0$ (represented by the blue bar), is fed to the quantizer, which works iteratively. First, the closest vector to $\mathbf{r}_0$ is found in the first level codebook. Let this closest vector be $\mathbf{e}_{c_0}$ (represented by the red bar). Then, the residual error is computed as $\mathbf{r}_1 := \mathbf{r}_0 - \mathbf{e}_{c_0}$. This is fed into the second level of the quantizer, and the process is repeated: The closest vector to $\mathbf{r}_1$ is found in the second level, say $\mathbf{e}_{c_1}$ (represented by the green bar), and then the second level residual error is computed as $\mathbf{r}_2 = \mathbf{r}_1 - \mathbf{e}_{c_1}$. Then, the process is repeated for a third time on $\mathbf{r}_2$. The semantic codes are computed as the indices of $\mathbf{e}_{c_0}$, $\mathbf{e}_{c_1}$, and $\mathbf{e}_{c_2}$ in their respective codebooks. In the example shown in the figure, this results in the code (7, 1, 4).

Спасибо за
внимание